DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0607 Design and Analysis of Algorithms

ASSESSMENT – CO3

Brute-Force and Exhaustive-Search Techniques

Complete Assessment Report

Item	Details
Student name	RITHIKA G
Register number / Section	192571027
Course Outcome	**CO3- Module 2 – Live Seller Leaderboard (Quick Sort)**
Nature / Marks	Individual · 50 marks
GitHub repository	https://github.com/192571027/CSA0607-
Implementation language	Python 3.12 (matplotlib for graphs)

Module 2 – Live Seller Leaderboard (Quick Sort)

Case: 100,000 seller scores are re-sorted every few seconds. Between two refreshes only a few scores change, so the input is almost always **nearly sorted**. The current code is Quick Sort with the **first element as pivot**, and the leaderboard freezes during sale events.

Chosen algorithm: Quick Sort with a **median-of-three pivot** (plus insertion-sort cut-off and an introsort guard in production).

---

1. Divide, Conquer, Combine

Step	Quick Sort
Divide	Choose a pivot and *partition* the range so that every element left of the pivot is smaller and every element right is not smaller. The pivot lands in its final position p. Costs $\Theta(n)$ ($n - 1$ comparisons).
Conquer	Recursively sort A[lo..p−1] and A[p+1..hi].
Combine	Nothing – the work is done in place, so the array is already sorted.

Unlike Merge Sort, all the work is in the *divide* step, and the quality of the division depends entirely on the pivot.

---

2. One partition step on the sample scores

Sample: 42, 17, 63, 8, 55, 29; pivot = first element = **42**; i marks the end of the "smaller than pivot" zone.

j	A[j]	A[j] < 42 ?	action	array after the step
–	–	–	start, i = 0	42 17 63 8 55 29
1	17	yes	i = 1, swap A[1],A[1]	42 17 63 8 55 29
2	63	no	–	42 17 63 8 55 29
3	8	yes	i = 2, swap A[2],A[3]	42 17 **8 63** 55 29
4	55	no	–	42 17 8 63 55 29
5	29	yes	i = 3, swap A[3],A[5]	42 17 8 **29** 55 **63**
end			swap pivot A[0] ↔ A[3]	**29 17 8 | 42 | 55 63**

- The pivot 42 is now in its final index 3.
- Left part [29, 17, 8] (all < 42), right part [55, 63] (all ≥ 42).

- Comparisons: **5 = n − 1**.
- Complete sort of the sample: 8, 17, 29, 42, 55, 63 (9 comparisons in total).

---

3. Why the first-element pivot freezes the leaderboard

When the data is already (nearly) sorted, the first element is (nearly) the **minimum** (or the maximum if the board is kept in descending order). The partition then produces:

sizes of the two parts: 0 and $n - 1$ (instead of $\approx n/2$ and $n/2$)

Every call removes only **one** element, so:

- the recursion goes n levels deep → 100,000 nested calls (risk of **stack overflow**, and poor cache behaviour);
- the total work is $(n-1) + (n-2) + \ldots + 1 = n(n-1)/2$ comparisons – quadratic;
- a *nearly sorted* array is almost as bad as a fully sorted one: in the experiment below, 1 % disturbed positions still gave 99 % of the $n(n-1)/2$ worst case.

Ironically, the property that should make the job easy (little change since the last refresh) is the exact property that triggers Quick Sort's worst case with this pivot rule. And because refreshes happen every few seconds, the $O(n^2)$ cost is paid again and again, and is largest during sale events when sellers' scores move constantly.

---

4. Recurrences and solutions

Let T(n) be the number of key comparisons. Partitioning n elements costs $n - 1$.

Best case – pivot always the median

$T(1) = T(0) = 0$

$T(n) = 2\,T(n/2) + (n - 1)$

Same recurrence as Merge Sort: solution $\mathbf{T(n) = n \lg n - n + 1 = \Theta(n \log n)}$ (Master theorem case 2: $a = 2$, $b = 2$, $f(n) = \Theta(n)$).

Worst case – pivot always the minimum or maximum

$T(n) = T(n - 1) + T(0) + (n - 1) = T(n - 1) + (n - 1)$

Unrolling:

$T(n) = (n-1) + (n-2) + \ldots + 1 + T(1) = n(n-1)/2 = \Theta(n^2)$

Average case (random pivot)

$T(n) = (n-1) + (2/n)\,\Sigma\, T(i) \Rightarrow T(n) \approx 2n \ln n \approx 1.39\, n \lg n = \Theta(n \log n)$; exactly $2(n+1)H_n - 4n$.

---

5. Estimate for 100,000 nearly sorted scores

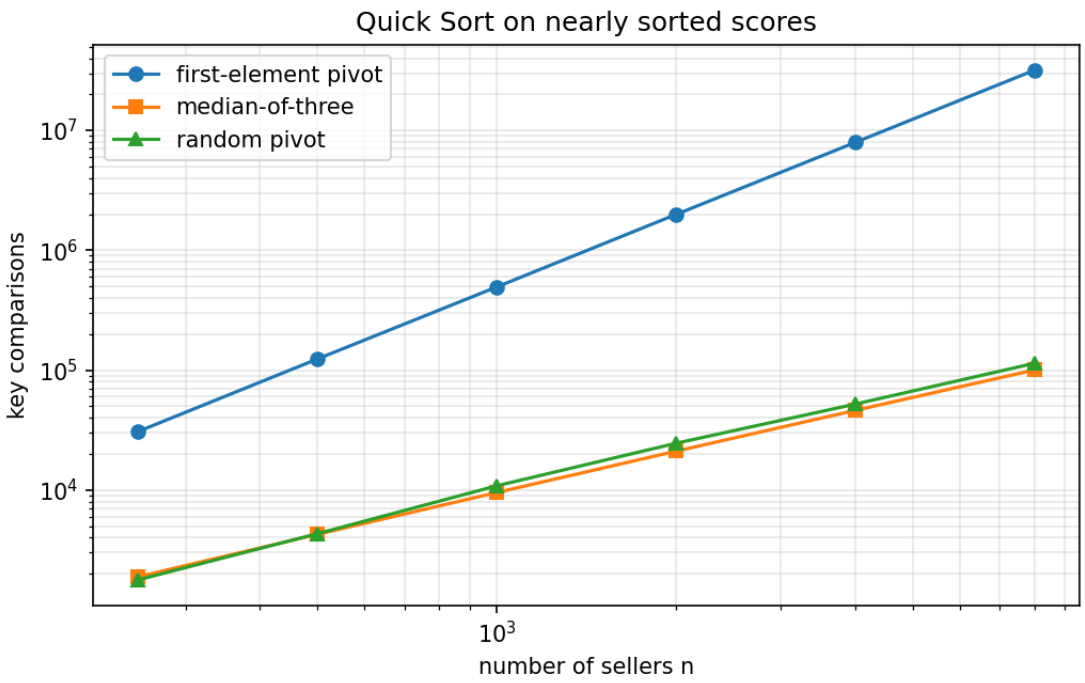
Strategy	Formula	Comparisons
First-element pivot (worst case)	$n(n-1)/2 = 100{,}000 \times 99{,}999 / 2$	**4,999,950,000 ≈ 5.0 × 10^9**
Median-of-three (sorted/nearly sorted ⇒ near-perfect splits)	$n \lg n - n + 1 = 100{,}000 \times 16.6096 - 100{,}000 + 1$	**≈ 1,560,965 ≈ 1.6 × 10^6**
Random pivot (expected)	$2(n+1)H_n - 4n$, $H_n \approx \ln n + 0.5772 = 12.0901$	≈ 2,018,052 ≈ 2.0 × 10^6

Improvement from first-element pivot to median-of-three: 4,999,950,000 / 1,560,965 ≈ **3,200 × fewer comparisons**. Recursion depth falls from 99,999 to about $\lg n \approx 17$.

Experimental confirmation (1 % of positions disturbed by adjacent swaps)

n	first-element	median-of-three	random	n(n−1)/2	n lg n	depth (first)	depth (median-of-3)
250	30,830	1,892	1,784	31,125	1,991	248	9
1,000	493,795	9,532	10,842	499,500	9,966	990	11
4,000	7,916,984	46,114	52,044	7,998,000	47,863	3,960	13
8,000	31,692,291	100,201	113,926	31,996,000	103,726	7,920	14

- The first-element column tracks $n(n-1)/2$ almost exactly (quadratic).
- The median-of-three column tracks $n \lg n$ (≈ 0.97 × $n \lg n$ here).
- Doubling n multiplies the first-element cost by ≈ 4 and the median-of-three cost by ≈ 2.2.



Quick sort comparisons

---

6. Improved pivot strategy

Proposal: median-of-three – use the median of A[lo], A[mid], A[hi] as the pivot (3 extra comparisons per partition, then swap it to A[lo] and use the same partition).

Why it works for this workload. * In a (nearly) sorted range the middle element is close to the true median, so the two sub-problems have almost equal size – the best-case recurrence applies and the cost becomes Θ(n log n). * The first and last elements of a sorted range are its extremes, but the median of {first, middle, last} is the middle value, so an extreme is never chosen as the pivot. * The 3 extra comparisons per partition add only a small constant (≈ 3 × number of partitions) – negligible against n. * It is simple to add to the existing code (a handful of lines), and it is deterministic and cache-friendly.

Alternatives considered.

Strategy	Nearly sorted input	Adversarial input	Remark
First element (current)	Θ(n²)	Θ(n²)	what is deployed
Random	Θ(n log n) expected, ≈	cannot be forced	needs a random number per partition; slightly more

Strategy	Nearly sorted input	Adversarial input	Remark
pivot	2.0×10^6 for n = 100,000		comparisons than median-of-3 (≈ 1.39 vs 1.19 n lg n on random data)
Median-of-three	**Θ(n log n)**	still possible (a crafted "median-of-3 killer" input)	**chosen: best on sorted/nearly sorted data**

Honest limitation found in testing. Median-of-three is not a guarantee: on fully *reverse-sorted* input with this Lomuto partition my implementation needed 504,005 comparisons for n = 2,000 (≈ $n^2/8$, versus 21,033 for sorted input), because the first partition scrambles the reverse order and later median choices become poor. Random pivot gave 26,504 on the same input. So for production, median-of-three should be hardened:

1. **Introsort guard** – count recursion depth and switch to Heap Sort when it exceeds 2 lg n, giving a guaranteed Θ(n log n) worst case.
2. **Recurse on the smaller side, loop on the larger** – stack depth ≤ lg n, so no stack overflow.
3. **Insertion-sort cut-off** (ranges of ≤ 16 elements) – fast for the nearly sorted sub-ranges.
4. **Three-way partitioning** if many sellers have equal scores.

Trade-offs. Quick Sort is **not stable** (not a problem for scores, since equal scores need no particular order – ties can be broken by seller ID if a fixed order is desired), and it gives no worst-case guarantee on its own. In exchange it sorts **in place** (O(1) extra memory apart from the O(log n) stack) and has the smallest constant factor of the Θ(n log n) sorts.

Beyond divide-and-conquer. Because only a few scores change between refreshes, an *adaptive* method such as insertion sort (cost n + number of inversions) or Timsort can finish in close to linear time. If the share of changed scores is tiny, use that instead (see the change condition in the final design).